RELATÓRIO TÉCNICO

Relatório de Auditoria de Segurança

AtendeMente

31 de agosto de 2026

Versão auditada **1.0.19** · commit **6ed2d6a**

Escopo auditado

Auditoria de código, estática e manual, do repositório completo na revisão indicada: backend Rust/Axum, frontend React, camada de persistência SQLite e migrations, configuração do Tauri, workflow de CI e o histórico do git. **Não** fazem parte do escopo: teste dinâmico contra uma instalação em execução, análise de dependências de terceiros (npm e crates), engenharia reversa dos binários publicados e revisão da infraestrutura de qualquer serviço externo.

Stack detectada

Camada	Tecnologia
Desktop / shell	Tauri v2 (Rust), janela única, capabilities restritas a core / shell-open / dialog
Frontend	React 18 + TypeScript + Vite 6 + Tailwind 3, validação com Zod 4
Backend	Rust + Axum 0.8 embarcado no processo Tauri (também compilável como **bin/server**)
Persistência	SQLite via SQLx 0.8 — um banco de autenticação e um banco por usuário
Autenticação	Local: Argon2id + token de sessão UUIDv4 guardado como SHA-256, TTL de 7 dias
Criptografia	AES-256-GCM com chave por usuário derivada por HKDF-SHA256 de um pepper mestre
Cofre de chaves	keyring v3 (Credential Manager / Keychain / Secret Service)
Deploy / CI	GitHub Actions (quality + build + release). Não há Docker, Helm nem Terraform

Nota metodológica

As cinco categorias pedidas pressupõem uma aplicação web multi-inquilino. O AtendeMente é um aplicativo desktop monousuário com API embarcada em loopback, então cada categoria foi remapeada para o equivalente desta stack antes de ser aplicada. Onde a categoria não tem correspondente, isso está dito explicitamente em vez de gerar achado forçado.

1. Banco sem tranca

O projeto não usa Supabase, logo não há RLS. O mecanismo de isolamento equivalente foi identificado como duplo: (a) um arquivo SQLite por usuário, em **<DATA_DIR>/<user_id>/atendemente.db** (config.rs:321), e (b) uma coluna **user_id** presente em toda tabela de dados e exigida em toda cláusula WHERE. A auditoria extraiu **todos** os literais SQL do crate e classificou cada um pela presença de escopo de dono, revisando manualmente os que não o tinham.

2. Permissão definida no navegador

O app é monousuário por instalação: não existem papéis (admin/editor). A categoria foi remapeada para **divergência de regra entre cliente e servidor** — toda validação ou gate feito em React foi cruzado com o handler Axum correspondente, para verificar se o servidor reimpõe a mesma regra.

3. IDOR

Percorridos os **34** registros de rota de **api/routes.rs**, as **9** rotas de **auth/mod.rs** e os **6** comandos Tauri de **commands.rs** e **main.rs** — handler por handler, não por amostragem — seguindo cada identificador recebido por path, query ou body até a consulta SQL que o consome.

4. Chaves expostas

Varredura do código-fonte, de **tauri.conf.json**, **Cargo.toml**, **package.json**, workflows do GitHub Actions, scripts e documentação. O **histórico completo do git** foi varrido com **git log -S** por padrões de segredo, e verificou-se com **git merge-base** a alcançabilidade de cada commit encontrado a partir de **main**.

5. Inputs sem tratamento (XSS)

Busca por sinks no frontend (innerHTML, dangerouslySetInnerHTML, eval, new Function, insertAdjacentHTML, document.write), por URLs de origem no usuário em href/src, e no backend por entrada do usuário chegando a HTML, cabeçalhos HTTP ou CSV sem escape. Como o produto é uma webview, as **duas** políticas de CSP do projeto — a do servidor e a da configuração do Tauri — foram comparadas entre si.

Cobertura da revisão

Superfície	Volume	Cobertura
Rotas de dados (api/routes.rs)	34 registros de rota	**Todos revisados**
Rotas de autenticação (auth/mod.rs)	9 rotas	**Todas revisadas**
Comandos Tauri (commands.rs, main.rs)	6 comandos	**Todos revisados**
Arquivos Rust do backend	31 arquivos, ~12.300 linhas	**Todos lidos ou varridos**
Consultas SQL literais	Extração automática + revisão manual	**100% classificadas**
Migrations	14 de dados + 5 de autenticação	**Todas lidas**
Arquivos do frontend	35 arquivos .ts/.tsx em src/	**Varredura por sinks + leitura dirigida**
Configuração e deploy	tauri.conf.json, capabilities, vite, CI	**Todos lidos**
Histórico do git	Todos os commits alcançáveis	**Varrido com git log -S**

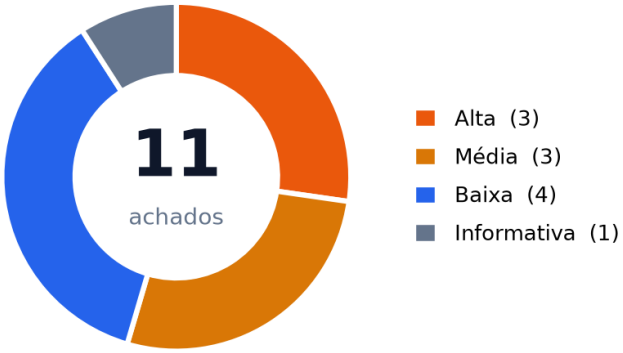
Nenhum achado deste relatório é especulativo: cada um foi verificado no código real, com arquivo e linha. Onde a exploração depende de uma condição externa — as regras de um projeto Firebase, a existência de uma segunda conta — a condição está declarada no próprio achado.

Resumo executivo

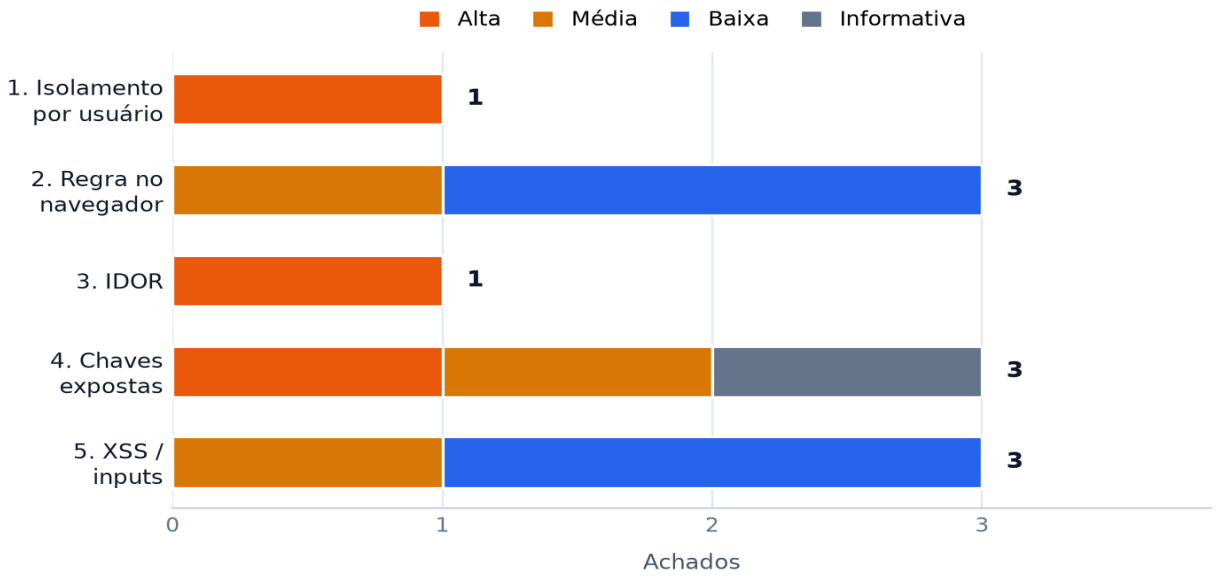
A revisão encontrou **11 achados**, dos quais **10** são acionáveis e **1** é informativo. **Nenhum achado crítico.** Os **3** de severidade alta concentram-se em dois pontos: a proteção dos dados em repouso (telefone e e-mail que escapam do envelope criptográfico) e o backup — que reúne, num único arquivo feito para ser copiado, o banco inteiro, os anexos decifrados e a chave-raiz da instalação. O restante do sistema resistiu bem: o isolamento por usuário está aplicado em todas as consultas, nenhum IDOR foi encontrado nos 49 pontos de entrada revisados, o SQL é integralmente parametrizado e o frontend não tem um único sink de XSS.



Distribuição por severidade



Achados por categoria auditada



Pontos fortes

Registrado com evidência de arquivo e linha, tanto para documentar o que está protegido quanto para demonstrar a cobertura desta auditoria: onde não há achado, é porque o ponto foi olhado e passou.

✓ Isolamento por usuário aplicado em 100% das consultas de dados

Extraímos todos os literais SQL do crate e conferimos um a um. Cada tabela de dados (patients, appointments, payments, session_records, record_files, recurring_series, backup_config, audit_logs) carrega **user_id**, e toda consulta de leitura, escrita, agregação e exportação inclui esse filtro. Exemplos: patients.rs:333, appointments.rs:47, payments.rs:125, dashboard.rs:36-81, exports.rs:87, files.rs:195. Isso soma-se ao isolamento físico de um arquivo SQLite por usuário (config.rs:321-325).

✓ Os 34 handlers de dados autenticam e derivam o banco do token, nunca do cliente

Nenhum handler de **api/routes.rs** aceita um **user_id** vindo do cliente. O padrão é invariável: **get_authenticated_user(&headers, &state)** (routes.rs:50-73) e em seguida **state.get_or_open_user_db(&user.id)**. Os 6 comandos Tauri seguem o mesmo caminho por **validate_token_and_get_db** (commands.rs:22-36).

✓ Verificação de posse presente em cada operação por ID — nenhum IDOR encontrado

Percorridos handler por handler. **update_patient** e **set_patient_status** abrem com **get_patient_detail(db, user_id, id)**, que devolve 404 para objeto alheio (patients.rs:688 e 765); **cancel_recurring_series** checa **rows_affected() == 0** (appointments.rs:493); as oito consultas de **files.rs** usam **WHERE id = ? AND user_id = ?**; e **create_upload_session** valida ainda o vínculo consulta↔paciente e pagamento↔consulta (files.rs:111-139).

✓ SQL integralmente parametrizado — nenhuma injeção possível

Não há uma única interpolação de entrada do usuário em SQL. Os dois lugares que montam SQL por formatação usam material controlado pelo próprio servidor: **VACUUM INTO** com um caminho temporário gerado internamente e escapado (backup.rs:108 e 835-837), e os nomes de tabela do restore, tirados da constante **RESTORE_TABLE_ORDER** (backup.rs:465-467) — com o comentário explícito "Only names from this list are ever interpolated into SQL".

✓ Frontend React sem nenhum sink de XSS

Zero ocorrências de **innerHTML**, **dangerouslySetInnerHTML**, **eval**, **new Function**, **insertAdjacentHTML** ou **document.write** em todo **src/**. Não há renderização de markdown nem de HTML. A única URL montada com dado do usuário é o link do WhatsApp, e ela passa por **replace(/\D/g, "")**, que descarta tudo que não for dígito e torna **javascript:** impossível (PatientDetail.tsx:101). O link do Meet é estático e leva **rel="noopener noreferrer"** (PatientDetail.tsx:84).

✓ Upload de anexos validado em três camadas independentes

Whitelist de extensões e lista de bloqueio (files.rs:12-13 e 46-57); coerência entre MIME declarado e extensão; e conferência dos **magic bytes** reais via crate **infer** (files.rs:59-100). O nome é normalizado contra travessia de diretório e caracteres de controle (files.rs:15-36), o caminho de destino exige que user, patient e appointment sejam UUIDs válidos (config.rs:334-339), e o conteúdo é cifrado com AES-256-GCM antes de tocar o disco (files.rs:210-217).

✓ Restauração de backup fecha zip-slip e recusa bundle de outro usuário

safe_join rejeita caminho absoluto, **..**, **** e **:** (backup.rs:823-833); **validate_hashes** aplica a mesma checagem e ainda confere o SHA-256 de cada entrada contra o manifesto (backup.rs:435-450); e **manifest.user_id != user_id** aborta a restauração (backup.rs:252). A restauração inteira é preparada em área de staging e só então trocada, com rollback dos anexos em caso de falha (backup.rs:794-821).

✓ Autenticação construída com cuidado acima da média

Argon2id para senha (auth_service.rs:64-79); token de sessão nunca armazenado em claro, só o SHA-256 (auth_service.rs:104-108); comparação de tempo constante do código de recuperação (auth_service.rs:27-37); hash dummy no login para não vazar existência de conta por tempo de resposta (auth_service.rs:16-23 e 243-247); código de recuperação de uso único, rotacionado no reset (auth_service.rs:463-478); e rate limit em register (3/h), login (5/10min), recover (3/15min), reset-password (5/15min) e unlock (5/10min).

✓ O bloqueio de tela é imposto pelo servidor, não pela interface

lock_handler apaga a chave AES do cache do processo e fecha o pool do banco do usuário (auth/mod.rs:335-336); o **unlock** exige senha verificada contra o hash Argon2 e passa por rate limit (auth/mod.rs:366-383). O **/auth/me** informa **locked** ao cliente, para evitar que a tela abra mostrando PII vazia como se fosse dado real (auth/mod.rs:237). O logout também limpa a chave, não só o lock (auth/mod.rs:207).

✓ Superfície de rede reduzida ao mínimo

O servidor liga-se incondicionalmente a **127.0.0.1** e **[::1]** (lib.rs:407 e 418), e a regra de firewall da antiga função de acesso móvel é removida na primeira execução. O CORS usa predicado de origem em vez de **Any** (lib.rs:93-129 e 375-387), com **allow_credentials(false)**. O servidor de estáticos fecha travessia de diretório com **canonicalize** + **starts_with** (lib.rs:314-339).

✓ Erros internos não vazam detalhe de implementação

A variante **Internal** de **AppError** é substituída por uma mensagem genérica antes de virar resposta (errors.rs:135-145), com testes dedicados (**internal_errors_are_sanitized**, **database_errors_are_sanitized**, **crypto_errors_are_sanitized**). Os cabeçalhos de segurança são aplicados com **layer** e não **route_layer**, de forma a cobrir também o fallback que serve o index.html (lib.rs:367-370).

✓ Exportação CSV protegida contra injeção de fórmula

escape_csv prefixa com aspa simples todo campo iniciado por **=**, **+**, **-**, @, tab ou CR, neutralizando a execução de fórmulas ao abrir o arquivo em Excel ou LibreOffice (exports.rs:44-66) — um vetor que passa despercebido na maioria das bases que exportam dado digitado pelo usuário.

Pontos fracos

Os riscos centrais, agrupados por tema em vez de por achado individual.

! A criptografia de PII tem um furo declarado

Telefone e e-mail ficam legíveis no índice de busca (F1). Quem obtiver o arquivo .db obtém os dados de contato de todos os pacientes sem precisar do pepper — o que reduz bastante o valor prático do envelope AES para esses dois campos.

! O backup concentra risco demais num arquivo só

Ele reúne o banco completo, os anexos decifrados e a chave-raiz da instalação (F3), e o servidor aceita protegê-lo com 8 caracteres (F4). É o artefato mais exposto do sistema — feito para ser copiado — e o que carrega mais segredo.

! Um recurso por usuário escapou do modelo de isolamento

A senha dos backups automáticos é global e as rotas que a gerenciam ignoram quem chamou (F2). É a única exceção encontrada num modelo que, no resto, é aplicado com rigor.

! A defesa contra XSS mais forte não é a que roda no produto

A CSP da webview é mais permissiva que a do servidor (F6) e não há biblioteca de sanitização no projeto. Hoje não existe sink, mas a rede de proteção está desligada justamente onde o usuário final executa o app.

! Regras de entrada moram no cliente

Tamanho mínimo de senha de backup (F4) e limites de tamanho de campo (F9) são impostos em TypeScript e não reimpostos em Rust. Qualquer chamador fora da interface os contorna.

Achados detalhados

Visão consolidada. Cada achado é desenvolvido logo em seguida, com o trecho de código, a razão pela qual é explorável, o impacto e as condições de exploração.

1. Banco sem tranca (isolamento por usuário)

Severidade	Arquivo:linha	Descrição

Alta	`src-tauri/src/db/migrations/20240101000009_search_tokens.sql:1-5`
	`src-tauri/src/features/patients.rs:236-252`
	`src-tauri/src/features/patients.rs:257-277`
		F1 — Telefone e e-mail dos pacientes ficam em texto claro no índice de busca, numa tabela que é a única sem escopo de dono

2. Regra definida no navegador

Severidade	Arquivo:linha	Descrição

Média	`src/pages/Settings.tsx:55`
	`src/pages/Settings.tsx:104`
	`src/pages/Settings.tsx:154`
	`src-tauri/src/features/backup.rs:25`
	`src-tauri/src/features/backup.rs:88-95`
	`src-tauri/src/config.rs:12`
		F4 — Tamanho mínimo da senha de backup: 12 caracteres no navegador, 8 no servidor

Baixa	`src-tauri/src/auth/mod.rs:255-268` `src-tauri/src/auth/auth_service.rs:357-365`	**F7** — Enumeração de contas em /auth/recover: a busca por e-mail acontece antes do rate limit e a mensagem distingue conta existente de inexistente

Baixa	`src-tauri/src/db/models.rs:1-375`
	`src-tauri/Cargo.toml:56`
	`src/lib/schemas.ts:14-24`
		F9 — Nenhum limite de tamanho de campo é imposto no servidor: a crate validator está declarada mas nenhum modelo a utiliza

3. IDOR (objeto acessado sem verificar o dono)

Severidade	Arquivo:linha	Descrição

Alta	`src-tauri/src/config.rs:9`
	`src-tauri/src/config.rs:89-119`
	`src-tauri/src/api/routes.rs:190`
	`src-tauri/src/api/routes.rs:204`
	`src-tauri/src/api/routes.rs:215`
	`src-tauri/src/lib.rs:237`
		F2 — A senha dos backups automáticos vive num slot global do cofre, e as três rotas que a gerenciam descartam o usuário autenticado

4. Chaves expostas

Severidade	Arquivo:linha	Descrição

Alta	`src-tauri/src/features/backup.rs:61`
	`src-tauri/src/features/backup.rs:128-137`
	`src-tauri/src/features/backup.rs:148`
	`src-tauri/src/crypto.rs:46-52`
		F3 — O pepper mestre — chave-raiz que deriva a chave AES de todos os usuários da máquina — é gravado dentro do arquivo de backup

Severidade	Arquivo:linha	Descrição
Média	`src/lib/auth.ts — commit ad4260f:1-6` `src/lib/auth.ts — commit ac6b0c7:1-6`	**F5** — Chave de API e identificadores de projeto Firebase permanecem no histórico do git

Informativa	`.github/workflows/ci.yml:102`
	`.github/workflows/ci.yml:136`
	`e2e/fixtures/auth.ts:21`
	`e2e/fixtures/test-data.ts:89`
	`scripts/seed-demo.ts — commit a05f804:25`
		F11 — Credenciais de teste e um pepper zerado em CI, scripts e fixtures

5. Inputs sem tratamento (XSS)

Severidade	Arquivo:linha	Descrição

Média	`src-tauri/tauri.conf.json:25`
	`src-tauri/src/middleware.rs:29-43`
	`src-tauri/src/middleware.rs:84-104`
		F6 — A CSP que vale para a webview permite script-src 'unsafe-inline' — a do servidor, mais estrita, não é a que protege o app instalado

Baixa	`src-tauri/src/api/routes.rs:602`
	`src-tauri/src/features/files.rs:166-183`
	`src-tauri/src/features/files.rs:292`
	`src-tauri/src/features/files.rs:320-328`
		F8 — O Content-Type do download vem do MIME declarado no upload, que só é validado se o upload for confirmado

Baixa	`src/style.css:1` `src-tauri/tauri.conf.json:25`	**F10** — O app busca fontes na internet a cada abertura, e a CSP foi afrouxada para permitir

Análise achado a achado

Alta

F1 · Telefone e e-mail dos pacientes ficam em texto claro no índice de busca, numa tabela que é a única sem escopo de dono

1. Banco sem tranca (isolamento por usuário)

src-tauri/src/db/migrations/20240101000009_search_tokens.sql:1-5 · src-tauri/src/features/patients.rs:236-252 · src-tauri/src/features/patients.rs:257-277

SITUACAO ATUAL

Corrigido — migration 15 substitui os valores em claro por HMAC-SHA256 de n-gramas, com chave derivada do pepper, e a tabela ganhou `user_id`.

```
-- migrations/20240101000009_search_tokens.sql:1-5
CREATE TABLE IF NOT EXISTS patient_search_tokens (
    patient_id TEXT NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    token_type TEXT NOT NULL,
    token_text TEXT NOT NULL          -- <- sem user_id, e o valor vai em claro
);

// features/patients.rs:236-252
if let Some(phone) = &pii.phone {
    let digits: String = phone.chars().filter(|c| c.is_ascii_digit()).collect();
    if !digits.is_empty() {
        tokens.push((patient_id.to_string(), "phone".to_string(), digits));
    }
}
if let Some(email) = &pii.email {
    let lower = email.to_lowercase();
    tokens.push((patient_id.to_string(), "email".to_string(), lower.clone()));
```

Por que é explorável

A migration 20240101000008 moveu a PII do paciente para um blob AES-256-GCM (**pii_encrypted**) e as colunas em claro passaram a ser gravadas como NULL. A migration 9 devolveu telefone e e-mail ao disco em texto claro, agora em outra tabela, para permitir busca por substring. O resultado é que os dois campos de contato mais identificáveis do prontuário estão legíveis para quem abrir o arquivo .db com qualquer cliente SQLite — sem senha, sem o pepper, sem passar pelo cofre de credenciais do sistema operacional. E o arquivo .db é justamente o que sai da máquina: backups em pen drive, pastas sincronizadas com a nuvem, um HD retirado de um notebook roubado. Além disso, **patient_search_tokens** é a única tabela de dados sem coluna **user_id**: o escopo de dono existe apenas indiretamente, pelo JOIN com **patients**. Hoje isso não vaza — as consultas de busca em patients.rs:360-396 prendem o resultado com um WHERE user_id externo — mas a proteção passa a depender de todo autor futuro lembrar de escrever o JOIN.

IMPACTO

Exposição de dados pessoais de pacientes (LGPD, art. 46) a partir de uma cópia do arquivo de banco, anulando na prática a criptografia de PII para telefone e e-mail.

CONDIÇÕES DE EXPLORAÇÃO

Não requer nenhuma configuração especial. Explorável por qualquer um com acesso de leitura ao arquivo **<DATA_DIR>/<user_id>/atendemente.db**.

Correção sugerida

Guardar um token derivado irreversível em vez do valor: por exemplo HMAC-SHA256 truncado por n-grama do telefone e do e-mail, com chave derivada do pepper. A busca por substring passa a gerar os mesmos n-gramas a

partir do termo consultado. Se a busca em claro for considerada requisito inegociável, o mínimo é adicionar a coluna **user_id** à tabela e ao WHERE, para que o isolamento não dependa do JOIN.

Alta

F2 · A senha dos backups automáticos vive num slot global do cofre, e as três rotas que a gerenciam descartam o usuário autenticado

3. IDOR (objeto acessado sem verificar o dono)

src-tauri/src/config.rs:9 · src-tauri/src/config.rs:89-119 · src-tauri/src/api/routes.rs:190 · src-tauri/src/api/routes.rs:204 · src-tauri/src/api/routes.rs:215 · src-tauri/src/lib.rs:237

SITUACAO ATUAL

Corrigido — a entrada do cofre passou a incluir o `user_id`, as tres rotas usam o usuario autenticado e a senha legada e migrada no boot.

```
// config.rs:9 — conta fixa, sem user_id no nome
const KEYCHAIN_BACKUP_PASSWORD: &str = "backup_password";

// config.rs:89-94
pub fn load_backup_password() -> Option<String> {
    Entry::new(KEYCHAIN_SERVICE, KEYCHAIN_BACKUP_PASSWORD)   // <- mesma entrada para todos
        .ok()
        .and_then(|entry| entry.get_password().ok())
// api/routes.rs:204 e 215 — o usuário é autenticado e em seguida descartado
async fn set_backup_password(...) {
    let _user = get_authenticated_user(&headers, &state).await?;
    crate::config::save_backup_password(&input.password).map_err(AppError::bad_request)?;

async fn clear_backup_password(...) {
    let _user = get_authenticated_user(&headers, &state).await?;
    crate::config::delete_backup_password().map_err(AppError::internal)?;

// lib.rs:237 — dentro de `for user_id in &users`
let Some(password) = config::load_backup_password() else { ... continue; };
```

Por que é explorável

Toda a aplicação foi construída em torno do isolamento por usuário — banco separado, chave AES separada, **user_id** em cada WHERE — e este recurso escapa dele. As três rotas de **/api/backup/password** autenticam o chamador e então jogam fora o **user.id**: o _ em **let _user** é a assinatura da falha, porque o alvo da operação não é derivado de quem chamou. Numa instalação com mais de uma conta — o app permite registrar quantas quiser, e o próprio scheduler itera sobre **SELECT id FROM auth_users** — isso produz três abusos concretos: (a) o usuário B chama **PUT /api/backup/password** e passa a conhecer a senha que cifra os backups automáticos **de A**, bastando obter o arquivo .atendemente para ler o prontuário completo de A; (b) o usuário B chama **DELETE** e os backups automáticos de A param de ser gerados em silêncio — lib.rs:237 apenas registra um warn e segue, enquanto a tela de A continua mostrando a frequência configurada; (c) A e B não conseguem manter senhas distintas: quem gravar por último vence.

IMPACTO

Quebra do isolamento entre contas para o artefato mais sensível do sistema (o backup contém o banco inteiro mais os anexos) e perda silenciosa de backup para o usuário que teve a senha removida por outro.

CONDIÇÕES DE EXPLORAÇÃO

Explorável quando a instalação tem mais de uma conta registrada. Com conta única o impacto se reduz, mas o defeito de projeto permanece e vira falha assim que a segunda conta é criada.

Correção sugerida

Tornar a entrada do cofre específica por usuário — por exemplo **Entry::new(KEYCHAIN_SERVICE, &format!("backup_password:{user_id}"))** — e propagar o **user.id** autenticado por **load/save/delete_backup_password** e pelo laço do scheduler em lib.rs. Trocar **let _user** por **let user** nos três handlers deixa o compilador cobrar o resto.

Alta

F3 · O pepper mestre — chave-raiz que deriva a chave AES de todos os usuários da máquina — é gravado dentro do arquivo de backup

4. Chaves expostas

src-tauri/src/features/backup.rs:61 · **src-tauri/src/features/backup.rs:128-137** · **src-tauri/src/features/backup.rs:148** · **src-tauri/src/crypto.rs:46-52**

SITUACAO ATUAL

Corrigido — formato de backup v3: o manifesto carrega um envelope da chave do usuario cifrado pela senha, e nao mais o pepper mestre. Bundles v1 e v2 continuam restaurando.

```
// features/backup.rs:128-137
let (salt_hex, pepper_field, pepper_fp) = if is_encrypted {
    let mut salt = [0u8; SALT_SIZE];
    rand::rngs::OsRng.fill_bytes(&mut salt);
    let fp = crypto::pepper_fingerprint().ok();
    let pepper_hex = crypto::get_pepper().map(|p| hex_encode(p));   // <- 32 bytes em hex
    // pepper_hex is only stored inside the encrypted ZIP (protected by password)
    (Some(hex_encode(&salt)), pepper_hex, fp)
// features/backup.rs:148 — vai para o manifest.json dentro do ZIP
    pepper: pepper_field,
// crypto.rs:46-52 — o que esse valor destranca
fn derive_key_inner(user_id: &str, pepper: &[u8; 32]) -> Result<[u8; 32], AppError> {
    let hk = Hkdf::<Sha256>::new(Some(pepper), user_id.as_bytes());
```

Por que é explorável

O pepper existe justamente para **não** sair da máquina: é gerado por CSPRNG na primeira execução e guardado no cofre de credenciais do sistema operacional (config.rs:146-229). Gravá-lo no backup transfere esse segredo para um arquivo feito para ser copiado — pen drive, e-mail, pasta sincronizada. O comentário da linha 133 reconhece o risco e o considera coberto pela cifragem do ZIP, mas isso rebaixa a proteção do pepper de "cofre do sistema operacional" para "a senha que o usuário escolheu", e o servidor aceita senha de **8 caracteres** nesse ponto (backup.rs:25 — ver F4). Como **derive_user_key** é apenas **HKDF(pepper, user_id)** e o **user_id** viaja em claro no mesmo manifesto, quem quebrar a senha do backup não obtém apenas os dados daquele usuário: obtém a chave de **todos** os usuários daquela instalação, e ela continua válida para os bancos que permanecem na máquina. O motivo funcional é legítimo — restaurar num computador novo, onde o cofre não existe mais — mas a solução escolhida embute a chave-raiz no artefato.

IMPACTO

O comprometimento de um único arquivo de backup, com a senha quebrada, expõe a chave-raiz de criptografia da instalação inteira — inclusive de contas cujo backup nunca vazou.

CONDIÇÕES DE EXPLORAÇÃO

Presente em todo backup cifrado: o campo só é preenchido quando **is_encrypted**. A exploração exige obter o arquivo e quebrar a senha escolhida pelo usuário.

Correção sugerida

Não transportar o pepper. Cifrar a chave por usuário com uma chave derivada da senha do backup (Argon2id) e guardar apenas esse envelope no manifesto, de modo que o segredo transportado seja função da senha e não a

chave-raiz da máquina. Enquanto isso não for feito: elevar o mínimo de senha (F4), manter o **pepper_fingerprint** — que já existe e basta para detectar divergência de chave — e remover o campo **pepper**, passando a exigir o pepper antigo do usuário na restauração entre máquinas.

Média

F4 · Tamanho mínimo da senha de backup: 12 caracteres no navegador, 8 no servidor

2. Regra definida no navegador

`src/pages/Settings.tsx:55` · `src/pages/Settings.tsx:104` · `src/pages/Settings.tsx:154` · `src-tauri/src/features/backup.rs:25` · `src-tauri/src/features/backup.rs:88-95` · `src-tauri/src/config.rs:12`

SITUACAO ATUAL

Corrigido — os dois caminhos de backup compartilham a constante de 12 caracteres.

```
// src/pages/Settings.tsx:55 — o que o usuário vê
if (pw.length < 12) {
  setPasswordError("Minimo 12 caracteres para backup com senha.");
  return;
}

// src-tauri/src/features/backup.rs:25 e 88-95 — o que o servidor aceita
const MIN_BACKUP_PASSWORD_LEN: usize = 8;
...
if let Some(pass) = password {
    if pass.chars().count() < MIN_BACKUP_PASSWORD_LEN {
        return Err(AppError::bad_request(format!(
            "A senha do backup deve ter no mínimo {} caracteres.", MIN_BACKUP_PASSWORD_LEN)));

// src-tauri/src/config.rs:12 — a senha automática, por outro lado, exige 12
pub const MIN_AUTO_BACKUP_PASSWORD_LEN: usize = 12;
```

Por que é explorável

A regra que o usuário enxerga é imposta apenas em React. Um **POST /api/backup** feito fora da interface — com curl, num script de automação, ou por qualquer código que passe a chamar a API depois — produz um backup válido protegido por 8 caracteres. E esse backup é o bundle completo: banco de pacientes, prontuários de sessão, anexos e, hoje, o pepper mestre (F3). A própria base de código já discorda de si mesma: **MIN_AUTO_BACKUP_PASSWORD_LEN = 12** em config.rs para a senha dos backups automáticos, contra **MIN_BACKUP_PASSWORD_LEN = 8** para o manual. E em nenhum dos dois caminhos há qualquer exigência além do comprimento.

IMPACTO

Backups contendo o prontuário completo podem ser gerados com senha muito mais fraca do que a aplicação promete, reduzindo o custo de um ataque offline sobre o arquivo — e, por tabela, sobre a chave-raiz que ele carrega.

CONDIÇÕES DE EXPLORAÇÃO

Explorável por qualquer chamada à API fora da interface. Não requer configuração insegura.

Correção sugerida

Elevar **MIN_BACKUP_PASSWORD_LEN** ao mesmo valor da constante de config.rs — ou, melhor, importar **MIN_AUTO_BACKUP_PASSWORD_LEN** nos dois caminhos, para que não voltem a divergir — e acrescentar uma checagem de qualidade além do comprimento.

Média

F5 · Chave de API e identificadores de projeto Firebase permanecem no histórico do git

4. Chaves expostas

src/lib/auth.ts — commit ad4260f:1-6 · src/lib/auth.ts — commit ac6b0c7:1-6

SITUACAO ATUAL

Resolvido pelo usuario — o projeto Firebase `atendemente-432ab` foi excluido, que era a correcao que de fato fechava o risco.

```
// src/lib/auth.ts:1-6 — em ad4260f e ac6b0c7, ambos ancestrais de main
const apiKey = import.meta.env.VITE_FIREBASE_API_KEY || "AIzaSyCMGKi2aHlRpOynxpjhb_7VZMMMqRaUa4Y";
const authDomain = import.meta.env.VITE_FIREBASE_AUTH_DOMAIN || "atendemente-432ab.firebaseapp.com";
const projectId = import.meta.env.VITE_FIREBASE_PROJECT_ID || "atendemente-432ab";
const storageBucket = import.meta.env.VITE_FIREBASE_STORAGE_BUCKET || "atendemente-432ab.firebasestorage.app";
const messagingSenderId = import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID || "416468948741";
const appId = import.meta.env.VITE_FIREBASE_APP_ID || "1:416468948741:web:f08ce79a49801f1fc3496c";
```

Por que é explorável

É o padrão exato de "default público que vira segredo real quando a variável não é sobrescrita": o **||** garante que a chave literal entra no bundle sempre que **VITE_FIREBASE_API_KEY** não estiver definida. O arquivo não existe mais no HEAD — a autenticação migrou para local, em Rust — mas **git log -S** confirma os dois commits e **git merge-base --is-ancestor** confirma que ambos são alcançáveis a partir de **main**: qualquer clone do repositório público os traz junto. Uma chave de API Web do Firebase é, por projeto, um identificador destinado ao cliente e não uma credencial de acesso — é por isso que este achado é médio e não crítico. Ainda assim ela expõe o projeto **atendemente-432ab** a abuso da API Identity Toolkit (criação de contas, consumo de cota) e a leitura de Firestore/Storage **na medida em que as regras de segurança daquele projeto forem permissivas** — o que não é verificável a partir deste repositório.

IMPACTO

Superfície residual num projeto Firebase que a aplicação atual não usa mais. Se o projeto seguir ativo com regras abertas, permite acesso não autenticado aos recursos dele.

CONDIÇÕES DE EXPLORAÇÃO

A exploração depende inteiramente das regras de segurança do projeto Firebase e de ele continuar ativo. A presença do segredo no histórico é fato verificado.

Correção sugerida

Verificar no console do Firebase se o projeto **atendemente-432ab** ainda existe. Se sim: revogar/rotacionar a chave, restringi-la por aplicativo e revisar as regras de Firestore e Storage; se o projeto não é mais usado, desativá-lo. Reescrever o histórico com **git filter-repo** é possível, mas num repositório público já clonado o valor deve ser tratado como comprometido de qualquer forma — desativar o projeto é a correção que de fato fecha o risco.

Média

F6 · A CSP que vale para a webview permite script-src 'unsafe-inline' — a do servidor, mais estrita, não é a que protege o app instalado

5. Inputs sem tratamento (XSS)

src-tauri/tauri.conf.json:25 · src-tauri/src/middleware.rs:29-43 · src-tauri/src/middleware.rs:84-104

SITUACAO ATUAL

Corrigido — `unsafe-inline` removido de `script-src` no tauri.conf.json, com teste que compara as duas politicas.

```
// src-tauri/tauri.conf.json:25 — a política que a webview do Tauri aplica
"csp": "default-src 'self'; connect-src 'self' http://localhost:* http://127.0.0.1:* ipc: asset: ws:;
        style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
        font-src 'self' https://fonts.gstatic.com; img-src 'self' data: blob:;
        script-src 'self' 'unsafe-inline' asset:"
                         ^^^^^^^^^^^^^^^^

// src-tauri/src/middleware.rs:37 — a política que o servidor envia
             script-src 'self'; \
             object-src 'none'; \
             base-uri 'none'; \
             form-action 'none'; \
```

Por que é explorável

O comentário em middleware.rs:25-28 registra corretamente que "a webview honra a CSP da configuração" — e é exatamente por isso que a política válida no produto instalado é a mais fraca das duas. Com **'unsafe-inline'** em **script-src**, qualquer injeção de HTML que venha a existir executa; sem ele, a mesma injeção seria inerte. A CSP da webview também não fecha **object-src**, **base-uri** nem **form-action**, que a do servidor fecha, e o teste **csp_locks_down_the_dangerous_directives** (middleware.rs:84-104) valida apenas a política do servidor — a divergência não é detectada por nenhum teste. Registre-se: **não foi encontrado nenhum sink de XSS no frontend** (zero ocorrências de innerHTML, dangerouslySetInnerHTML, eval ou new Function), de modo que isto é o enfraquecimento de uma defesa em profundidade, não uma injeção explorável hoje. O que a torna relevante é o conteúdo renderizado: nome, anotações, histórico clínico e medicações são texto livre digitado pelo usuário ou importado de CSV.

IMPACTO

A camada que conteria um XSS futuro está desativada precisamente no artefato distribuído aos usuários. Também não há nenhuma biblioteca de sanitização no projeto, caso um sink venha a ser introduzido.

CONDIÇÕES DE EXPLORAÇÃO

Enfraquecimento de defesa em profundidade: só produz impacto direto se um sink de HTML for introduzido no frontend.

Correção sugerida

Remover **'unsafe-inline'** de **script-src** no tauri.conf.json e alinhar as duas políticas, acrescentando **object-src 'none'**, **base-uri 'none'** e **form-action 'none'**. Estender o teste de middleware.rs para ler o tauri.conf.json e exigir das duas políticas o mesmo conjunto mínimo de diretivas, para que não voltem a divergir.

Baixa

F7 · Enumeração de contas em /auth/recover: a busca por e-mail acontece antes do rate limit e a mensagem distingue conta existente de inexistente

2. Regra definida no navegador

src-tauri/src/auth/mod.rs:255-268 · src-tauri/src/auth/auth_service.rs:357-365

SITUACAO ATUAL

Corrigido — o rate limit passou a ser aplicado sobre o identificador recebido, antes do lookup, e a resposta e a mesma para conta inexistente e codigo errado.

```
// auth/mod.rs:255-268 — o lookup por e-mail roda ANTES do enforce_rate_limit
let user_id = match (&input.user_id, &input.email) {
    (Some(uid), _) if !uid.is_empty() => uid.clone(),
    (_, Some(email)) if !email.is_empty() => {
        auth_service::find_user_id_by_email(&state.auth_db, email)
            .await
            .map_err(|e| AppError::not_found(e))?      // <- o 404 distingue a conta
    }
    _ => return Err(AppError::bad_request("Informe user_id ou email.")),
};

crate::rate_limit::enforce_rate_limit(          // <- só agora
    &state.auth_db, "auth:password-reset", &user_id, 3, 900_000,
).await?;

// auth_service.rs:364
.ok_or_else(|| "Nenhuma conta encontrada com este email.".to_string())
```

Por que é explorável

O limite de 3 tentativas por 15 minutos é aplicado sobre o **user_id**, que só existe depois de a conta ter sido encontrada. Consultas com e-mail inexistente, portanto, nunca incrementam contador nenhum: dá para varrer endereços indefinidamente e distinguir o 404 ("Nenhuma conta encontrada com este email.") do 401 ("Chave de recuperação inválida."). O contraste com o resto do arquivo é o que torna isto um achado e não uma escolha: o **login** foi cuidadosamente protegido contra o mesmo problema, gastando um Argon2 sobre um hash dummy quando a conta não existe (auth_service.rs:16-23 e 243-247) e devolvendo sempre "Email ou senha inválidos.". O caminho de recuperação não recebeu o mesmo tratamento. O servidor escuta apenas em loopback e o CORS por predicado impede que um site qualquer leia a resposta, o que mantém a severidade baixa.

IMPACTO

Confirmação de quais endereços de e-mail têm conta na máquina, sem custo e sem limite. Útil como etapa preparatória de um ataque local ao código de recuperação.

CONDIÇÕES DE EXPLORAÇÃO

Requer alcançar a API em 127.0.0.1:3001, isto é, código já em execução na máquina do usuário. O servidor não escuta na rede.

Correção sugerida

Aplicar o rate limit sobre o identificador **recebido** — o e-mail informado — antes de resolvê-lo, e devolver a mesma resposta para conta existente e inexistente, no mesmo espírito do que já é feito no login.

Baixa

F8 · O Content-Type do download vem do MIME declarado no upload, que só é validado se o upload for confirmado

5. Inputs sem tratamento (XSS)

src-tauri/src/api/routes.rs:602 · src-tauri/src/features/files.rs:166-183 · src-tauri/src/features/files.rs:292 · src-tauri/src/features/files.rs:320-328

SITUACAO ATUAL

Corrigido — o `Content-Type` do download e derivado dos magic bytes do conteudo decifrado.

```
// features/files.rs:179 — o MIME entra no banco como o cliente declarou
    .bind(&input.mime_type)

// features/files.rs:292 — a validação de magic bytes acontece só no confirm_upload
    validate_file_content(&data, &file.original_name, &file.mime_type)?;

// features/files.rs:320-323 — o download não revalida; exige apenas deleted_at IS NULL
    r#"SELECT * FROM record_files WHERE id = ? AND user_id = ? AND deleted_at IS NULL"#
// api/routes.rs:602 — e o valor vai direto para o cabeçalho
    (HeaderName::from_static("content-type"), file.mime_type),
```

Por que é explorável

create_upload_session grava **input.mime_type** sem validar; quem valida o MIME contra a extensão e contra os magic bytes reais é **validate_file_content**, chamada apenas em **confirm_upload** (files.rs:292). Mas **download_file** não exige confirmação — a única condição é **deleted_at IS NULL**. Uma sessão de upload criada e escrita, porém nunca confirmada, fica baixável com um **Content-Type** arbitrário escolhido pelo chamador. Três coisas seguram o impacto, e por isso a severidade é baixa: o **Content-Disposition: attachment** na linha seguinte impede a renderização inline; a extensão já foi restrita a pdf/docx/png/jpg/jpeg em **validate_file_metadata** (files.rs:46-57); e o app é monousuário, de modo que atacante e vítima são a mesma conta. O defeito real é de consistência: o modelo do código presume que todo registro legível passou pela validação, e há um caminho em que isso não vale.

IMPACTO

Um registro de arquivo não confirmado é servido com cabeçalho controlado pelo chamador. Sem impacto direto hoje, graças ao **attachment** e à whitelist de extensões.

CONDIÇÕES DE EXPLORAÇÃO

Requer uma sessão autenticada da própria conta e um upload deixado deliberadamente sem confirmar.

Correção sugerida

Marcar a confirmação no banco — uma coluna **confirmed_at** — e exigi-la em **download_file** e **list_files_by_appointment**. Alternativa mais simples: ignorar o MIME armazenado na resposta e derivar o **Content-Type** dos magic bytes do conteúdo decifrado, no momento do download.

Baixa

F9 · Nenhum limite de tamanho de campo é imposto no servidor: a crate validator está declarada mas nenhum modelo a utiliza

2. Regra definida no navegador

src-tauri/src/db/models.rs:1-375 · src-tauri/Cargo.toml:56 · src/lib/schemas.ts:14-24

SITUACAO ATUAL

Corrigido — os sete structs de entrada ganharam `#[validate]` e os handlers chamam `validate()`.

```
# src-tauri/Cargo.toml:56 — a dependência existe
validator = { version = "0.19", features = ["derive"] }

// src-tauri/src/db/models.rs — nenhuma ocorrência de #[validate] em todo o arquivo
pub struct CreatePatientInput {
    pub full_name: String,
    pub health_history: Option<String>,     // sem limite
    pub medications_in_use: Option<String>, // sem limite
    pub admin_notes: Option<String>,        // sem limite

// src/lib/schemas.ts:14-24 — a única validação de forma é a do cliente
export const patientSchema = z.object({
  full_name: z.string().min(2, "Nome deve ter no mínimo 2 caracteres"),
  health_history: z.string().optional(),
```

Por que é explorável

AppError tem uma variante **Validation** pronta para **ValidationErrors** (errors.rs:148-165) e a crate está no Cargo.toml, mas nenhum struct de entrada carrega atributos **#[validate]** — a validação de forma existe só em Zod, no cliente. Na prática, um **POST /api/patients** direto aceita um **admin_notes** de vários megabytes, limitado apenas pelo **DefaultBodyLimit** global de 20 MB (lib.rs:371). As regras que **de fato** importam para segurança estão no servidor e foram verificadas: nome com no mínimo 3 caracteres (patients.rs:620), senha com no mínimo 8 (auth_service.rs:121), fim depois do início (appointments.rs:225), recorrência limitada a 52 ocorrências (appointments_recurrence.rs:18-21). O que falta é o teto de tamanho.

IMPACTO

Crescimento descontrolado do banco e da memória por requisição, a partir de chamadas fora da interface. Sem impacto sobre confidencialidade ou integridade.

CONDIÇÕES DE EXPLORAÇÃO

Requer chamada à API fora da interface. O limite global de corpo de 20 MB continua valendo.

Correção sugerida

Aplicar **#[validate(length(max = ...))]** nos structs de entrada de **models.rs** e chamar **input.validate()?** no início de cada handler que os recebe — a variante de erro já está implementada e formatada.

Baixa

F10 · O app busca fontes na internet a cada abertura, e a CSP foi afrouxada para permitir

5. Inputs sem tratamento (XSS)

src/style.css:1 · src-tauri/tauri.conf.json:25

SITUACAO ATUAL

Corrigido — as fontes sao servidas do bundle e os dois dominios sairam da CSP.

```
/* src/style.css:1 */
@import url("https://fonts.googleapis.com/css2?family=Manrope:wght@400;500;600;700;800&
             family=Newsreader:ital,wght@0,400;0,500;0,600;0,700;1,400&display=swap");
// src-tauri/tauri.conf.json:25 — a CSP precisou ser aberta por causa disso
"style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
 font-src 'self' https://fonts.gstatic.com;"
```

Por que é explorável

Um aplicativo desktop de prontuário psicológico, que o README descreve como local, faz uma requisição a servidores do Google a cada abertura. O conteúdo do prontuário não vaza, mas o padrão de uso sim: endereço IP do consultório e horário de cada sessão de trabalho ficam registrados num terceiro que não é operador contratado para esse tratamento. É também a razão de a CSP da webview precisar liberar dois domínios externos (F6), e uma dependência de rede num app que deveria funcionar sem ela: sem internet, a tipografia cai para a fonte de sistema.

IMPACTO

Vazamento de metadados de uso para terceiro e dependência de rede desnecessária. Relevante para o registro de operadores exigido pela LGPD.

CONDIÇÕES DE EXPLORAÇÃO

Ocorre em toda execução com rede disponível. Não requer configuração especial.

Correção sugerida

Baixar os arquivos **.woff2** das duas famílias, servi-los do próprio bundle com **@font-face** local, e então remover **fonts.googleapis.com** e **fonts.gstatic.com** da CSP.

Informativa

F11 · Credenciais de teste e um pepper zerado em CI, scripts e fixtures

4. Chaves expostas

.github/workflows/ci.yml:102 · .github/workflows/ci.yml:136 · e2e/fixtures/auth.ts:21 · e2e/fixtures/test-data.ts:89 · scripts/seed-demo.ts — commit a05f804:25

SITUACAO ATUAL

Corrigido — peppers degenerados sao recusados no boot e o `hex_decode` do config nao entra mais em panico com entrada multi-byte.

```
# .github/workflows/ci.yml:102
export MASTER_PEPPER=0000000000000000000000000000000000000000000000000000000000000000

# .github/workflows/ci.yml:136
-d '{"email":"smoke@test.local","password":"senha123456","full_name":"Smoke"}'

// e2e/fixtures/auth.ts:21
const password = "test123456";
// scripts/seed-demo.ts:25 (arquivo já removido; permanece no commit a05f804)
const demoPassword = "AtendeMente123!";
```

Por que é explorável

Registrado por completude da varredura, não como falha. Nenhum desses valores é segredo de produção: o pepper zerado do CI roda contra bancos em **mktemp -d** e existe justamente para que o smoke test não dependa do cofre de credenciais do runner; as senhas de e2e criam contas descartáveis. O único ponto que merece registro é que **load_or_generate_pepper** (config.rs:146-180) aceita um pepper de 32 bytes nulos sem qualquer objeção — não existe validação de inicialização que recuse valores notoriamente fracos ou de exemplo. Vale também anotar o acerto correspondente: a variável de ambiente **nunca** é persistida no cofre (o comentário em config.rs:147-154 documenta a regressão que motivou a mudança), então um pepper de teste não pode mais substituir silenciosamente o real.

> **IMPACTO**
>
> Nenhum sobre a instalação do usuário final.

> **CONDIÇÕES DE EXPLORAÇÃO**
>
> Não aplicável.

Correção sugerida

Opcionalmente, recusar peppers degenerados — todos os bytes iguais — em **load_or_generate_pepper** quando o processo não estiver num modo de teste declarado, para que um valor de exemplo copiado da documentação não vire chave real.

Recomendações priorizadas

P1 é o que deve entrar na próxima versão: risco alto com correção de baixo custo. P2 exige migration, decisão de produto ou acesso a um console externo. P3 é endurecimento — vale fazer, não bloqueia release.

Prior.	Ação	Detalhe	Achado
P1	**Escopar a senha de backup por usuário**	Incluir o **user_id** na entrada do cofre e propagá-lo pelas três rotas e pelo scheduler. É a correção de menor custo entre as de severidade alta e fecha uma quebra direta de isolamento.	**F2**
P1	**Parar de transportar o pepper mestre dentro do backup**	Substituir o campo **pepper** por um envelope da chave do usuário cifrado com a senha do backup, mantendo o **pepper_fingerprint** para detecção de divergência.	**F3**
P1	**Unificar em 12 o mínimo de senha de backup no servidor**	Compartilhar uma única constante entre os dois caminhos de backup. Correção de uma linha, que sustenta diretamente a mitigação de F3.	**F4**
P2	**Remover telefone e e-mail em claro do índice de busca**	Migrar **patient_search_tokens** para tokens derivados irreversíveis por n-grama e adicionar a coluna **user_id** à tabela. Exige migration de reindexação e alteração da consulta de busca.	**F1**
P2	**Verificar e neutralizar o projeto Firebase exposto**	Confirmar se **atendemente-432ab** ainda existe; se sim, rotacionar a chave, restringi-la e revisar as regras de Firestore e Storage. Depende de acesso ao console, não do código.	**F5**
P2	**Alinhar as duas políticas de CSP e travar a divergência com teste**	Remover **'unsafe-inline'** de **script-src** no tauri.conf.json e estender o teste de middleware.rs para comparar as duas políticas.	**F6**
P3	**Fechar a enumeração de contas em /auth/recover**	Rate limit sobre o e-mail informado, antes de resolvê-lo, e resposta genérica — o mesmo tratamento que o login já recebeu.	**F7**
P3	**Exigir confirmação do upload no download, ou derivar o Content-Type do conteúdo**	Fecha o único caminho em que um registro de arquivo é servido sem ter passado pela validação de magic bytes.	**F8**
P3	**Ativar a crate validator nos modelos de entrada**	A variante de erro já existe e está formatada; falta declarar os limites e chamar **validate()** nos handlers.	**F9**
P3	**Empacotar as fontes localmente e fechar a CSP**	Elimina a requisição externa a cada abertura e permite remover dois domínios da política.	**F10**

Issues para o GitHub

Cada bloco abaixo é o texto completo de uma issue em Markdown, pronto para copiar e colar. Achados relacionados foram agrupados numa única issue quando compartilham a mesma correção, para não gerar ruído no backlog.

--- ISSUE 1 ---

````
## [Segurança] PII de contato em texto claro em patient_search_tokens

**Labels:** `security, alta, lgpd`
**Severidade:** Alta
**Categoria:** 1. Banco sem tranca (isolamento por usuário)

### Problema
A migration 20240101000008 moveu a PII do paciente para um blob AES-256-GCM (`pii_encrypted`) e as colunas em claro passaram a ser gravadas como NULL. A migration 9 devolveu telefone e e-mail ao disco em texto claro, agora em outra tabela, para permitir busca por substring. O resultado é que os dois campos de contato mais identificáveis do prontuário estão legíveis para quem abrir o arquivo .db com qualquer cliente SQLite — sem senha, sem o pepper, sem passar pelo cofre de credenciais do sistema operacional. E o arquivo .db é justamente o que sai da máquina: backups em pen drive, pastas sincronizadas com a nuvem, um HD retirado de um notebook roubado. Além disso, `patient_search_tokens` é a única tabela de dados sem coluna `user_id`: o escopo de dono existe apenas indiretamente, pelo JOIN com `patients`. Hoje isso não vaza — as consultas de busca em patients.rs:360-396 prendem o resultado com um WHERE user_id externo — mas a proteção passa a depender de todo autor futuro lembrar de escrever o JOIN.

### Evidência
- `src-tauri/src/db/migrations/20240101000009_search_tokens.sql:1-5`
- `src-tauri/src/features/patients.rs:236-252`
- `src-tauri/src/features/patients.rs:257-277`
```
-- migrations/20240101000009_search_tokens.sql:1-5
CREATE TABLE IF NOT EXISTS patient_search_tokens (
    patient_id TEXT NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
    token_type TEXT NOT NULL,
    token_text TEXT NOT NULL          -- <- sem user_id, e o valor vai em claro
);
// features/patients.rs:236-252
if let Some(phone) = &pii.phone {
    let digits: String = phone.chars().filter(|c| c.is_ascii_digit()).collect();
    if !digits.is_empty() {
        tokens.push((patient_id.to_string(), "phone".to_string(), digits));
    }
}
if let Some(email) = &pii.email {
    let lower = email.to_lowercase();
    tokens.push((patient_id.to_string(), "email".to_string(), lower.clone()));
```
### Impacto

Exposição de dados pessoais de pacientes (LGPD, art. 46) a partir de uma cópia do arquivo de banco, anulando na prática a criptografia de PII para telefone e e-mail.
**Condições de exploração:** Não requer nenhuma configuração especial. Explorável por qualquer um com acesso de leitura ao arquivo `<DATA_DIR>/<user_id>/atendemente.db`.

### Correção sugerida
Guardar um token derivado irreversível em vez do valor: por exemplo HMAC-SHA256 truncado por n-grama do telefone e do e-mail, com chave derivada do pepper. A busca por substring passa a gerar os mesmos n-gramas a partir do termo consultado. Se a busca em claro for considerada requisito inegociável, o mínimo é adicionar a coluna `user_id` à tabela e ao WHERE, para que o isolamento não dependa do JOIN.

### Critérios de aceite
````

```
- [ ] `patient_search_tokens.token_text` não contém mais telefone nem e-mail legíveis (verificável com
`SELECT token_text FROM patient_search_tokens` num banco de teste)
- [ ] A tabela tem coluna `user_id NOT NULL` e todas as consultas que a leem filtram por ela
- [ ] Busca por telefone e por e-mail continua funcionando (teste de integração cobrindo busca parcial)
- [ ] Existe migration que reindexa as linhas já gravadas em claro
---
_Aberto a partir do relatório de auditoria de segurança de 31 de agosto de 2026 (achado F1, versão 1.0.19,
commit 6ed2d6a)._
> **Situação:** Corrigido — migration 15 substitui os valores em claro por HMAC-SHA256 de n-gramas, com
chave derivada do pepper, e a tabela ganhou `user_id`.
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

````
## [Segurança] Senha de backup automático é global e ignora o usuário autenticado

**Labels:** `security, alta, idor`
**Severidade:** Alta
**Categoria:** 3. IDOR (objeto acessado sem verificar o dono)

### Problema
Toda a aplicação foi construída em torno do isolamento por usuário — banco separado, chave AES separada,
`user_id` em cada WHERE — e este recurso escapa dele. As três rotas de `/api/backup/password` autenticam o
chamador e então jogam fora o `user.id`: o `_` em `let _user` é a assinatura da falha, porque o alvo da
operação não é derivado de quem chamou. Numa instalação com mais de uma conta — o app permite registrar
quantas quiser, e o próprio scheduler itera sobre `SELECT id FROM auth_users` — isso produz três abusos
concretos: (a) o usuário B chama `PUT /api/backup/password` e passa a conhecer a senha que cifra os backups
automáticos `de A`, bastando obter o arquivo .atendemente para ler o prontuário completo de A; (b) o
usuário B chama `DELETE` e os backups automáticos de A param de ser gerados em silêncio — lib.rs:237 apenas
registra um warn e segue, enquanto a tela de A continua mostrando a frequência configurada; (c) A e B não
conseguem manter senhas distintas: quem gravar por último vence.

### Evidência
- `src-tauri/src/config.rs:9`
- `src-tauri/src/config.rs:89-119`
- `src-tauri/src/api/routes.rs:190`
- `src-tauri/src/api/routes.rs:204`
- `src-tauri/src/api/routes.rs:215`
- `src-tauri/src/lib.rs:237`

```
// config.rs:9 — conta fixa, sem user_id no nome
const KEYCHAIN_BACKUP_PASSWORD: &str = "backup_password";

// config.rs:89-94
pub fn load_backup_password() -> Option<String> {
    Entry::new(KEYCHAIN_SERVICE, KEYCHAIN_BACKUP_PASSWORD)   // <- mesma entrada para todos
        .ok()
        .and_then(|entry| entry.get_password().ok())
// api/routes.rs:204 e 215 — o usuário é autenticado e em seguida descartado
async fn set_backup_password(...) {
    let _user = get_authenticated_user(&headers, &state).await?;
    crate::config::save_backup_password(&input.password).map_err(AppError::bad_request)?;

async fn clear_backup_password(...) {
    let _user = get_authenticated_user(&headers, &state).await?;
    crate::config::delete_backup_password().map_err(AppError::internal)?;

// lib.rs:237 — dentro de `for user_id in &users`
let Some(password) = config::load_backup_password() else { ... continue; };
```

### Impacto
````

````
Quebra do isolamento entre contas para o artefato mais sensível do sistema (o backup contém o banco inteiro
mais os anexos) e perda silenciosa de backup para o usuário que teve a senha removida por outro.

**Condições de exploração:** Explorável quando a instalação tem mais de uma conta registrada. Com conta
única o impacto se reduz, mas o defeito de projeto permanece e vira falha assim que a segunda conta é
criada.
### Correção sugerida

Tornar a entrada do cofre específica por usuário — por exemplo `Entry::new(KEYCHAIN_SERVICE,
&format!("backup_password:{user_id}"))` — e propagar o `user.id` autenticado por
`load/save/delete_backup_password` e pelo laço do scheduler em lib.rs. Trocar `let _user` por `let user`
nos três handlers deixa o compilador cobrar o resto.
### Critérios de aceite

- [ ] A entrada do cofre inclui o `user_id` no nome da conta
- [ ] `load_backup_password`, `save_backup_password` e `delete_backup_password` recebem `user_id`
- [ ] Os três handlers de `/api/backup/password` usam o `user.id` do token, não `_user`
- [ ] O scheduler em `lib.rs` busca a senha do usuário da iteração
- [ ] Teste: com duas contas, definir a senha em A não altera nem remove a de B
---
_Aberto a partir do relatório de auditoria de segurança de 31 de agosto de 2026 (achado F2, versão 1.0.19,
commit 6ed2d6a)._

> **Situação:** Corrigido — a entrada do cofre passou a incluir o `user_id`, as tres rotas usam o usuario
autenticado e a senha legada e migrada no boot.
````

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

````
## [Segurança] Master pepper é serializado dentro do manifesto do backup

**Labels:** `security, alta, crypto`
**Severidade:** Alta
**Categoria:** 4. Chaves expostas

### Problema
O pepper existe justamente para `não` sair da máquina: é gerado por CSPRNG na primeira execução e guardado
no cofre de credenciais do sistema operacional (config.rs:146-229). Gravá-lo no backup transfere esse
segredo para um arquivo feito para ser copiado — pen drive, e-mail, pasta sincronizada. O comentário da
linha 133 reconhece o risco e o considera coberto pela cifragem do ZIP, mas isso rebaixa a proteção do
pepper de "cofre do sistema operacional" para "a senha que o usuário escolheu", e o servidor aceita senha
de `8 caracteres` nesse ponto (backup.rs:25 — ver F4). Como `derive_user_key` é apenas `HKDF(pepper,
user_id)` e o `user_id` viaja em claro no mesmo manifesto, quem quebrar a senha do backup não obtém apenas
os dados daquele usuário: obtém a chave de `todos` os usuários daquela instalação, e ela continua válida
para os bancos que permanecem na máquina. O motivo funcional é legítimo — restaurar num computador novo,
onde o cofre não existe mais — mas a solução escolhida embute a chave-raiz no artefato.

### Evidência
- `src-tauri/src/features/backup.rs:61`
- `src-tauri/src/features/backup.rs:128-137`
- `src-tauri/src/features/backup.rs:148`
- `src-tauri/src/crypto.rs:46-52`
```
// features/backup.rs:128-137
let (salt_hex, pepper_field, pepper_fp) = if is_encrypted {
    let mut salt = [0u8; SALT_SIZE];
    rand::rngs::OsRng.fill_bytes(&mut salt);
    let fp = crypto::pepper_fingerprint().ok();
    let pepper_hex = crypto::get_pepper().map(|p| hex_encode(p));   // <- 32 bytes em hex
    // pepper_hex is only stored inside the encrypted ZIP (protected by password)
    (Some(hex_encode(&salt)), pepper_hex, fp)

// features/backup.rs:148 — vai para o manifest.json dentro do ZIP
    pepper: pepper_field,

// crypto.rs:46-52 — o que esse valor destranca
fn derive_key_inner(user_id: &str, pepper: &[u8; 32]) -> Result<[u8; 32], AppError> {
````
````

````
    let hk = Hkdf::<Sha256>::new(Some(pepper), user_id.as_bytes());
```

### Impacto
O comprometimento de um único arquivo de backup, com a senha quebrada, expõe a chave-raiz de criptografia da instalação inteira — inclusive de contas cujo backup nunca vazou.

**Condições de exploração:** Presente em todo backup cifrado: o campo só é preenchido quando `is_encrypted`. A exploração exige obter o arquivo e quebrar a senha escolhida pelo usuário.
### Correção sugerida

Não transportar o pepper. Cifrar a chave por usuário com uma chave derivada da senha do backup (Argon2id) e guardar apenas esse envelope no manifesto, de modo que o segredo transportado seja função da senha e não a chave-raiz da máquina. Enquanto isso não for feito: elevar o mínimo de senha (F4), manter o `pepper_fingerprint` — que já existe e basta para detectar divergência de chave — e remover o campo `pepper`, passando a exigir o pepper antigo do usuário na restauração entre máquinas.
### Critérios de aceite

- [ ] `manifest.json` de um backup recém-criado não contém material de chave no campo `pepper`
- [ ] Restauração numa máquina com pepper diferente continua funcionando (teste de integração, reaproveitando as fixtures de `src-tauri/tests/fixtures/`)
- [ ] Backups já emitidos no formato antigo continuam restauráveis
- [ ] O `pepper_fingerprint` segue presente para detecção de divergência
---
_Aberto a partir do relatório de auditoria de segurança de 31 de agosto de 2026 (achado F3, versão 1.0.19, commit 6ed2d6a)._

> **Situação:** Corrigido — formato de backup v3: o manifesto carrega um envelope da chave do usuario cifrado pela senha, e nao mais o pepper mestre. Bundles v1 e v2 continuam restaurando.
````

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

````
## [Segurança] Mínimo de senha de backup diverge entre cliente (12) e servidor (8)

**Labels:** `security, media`
**Severidade:** Média
**Categoria:** 2. Regra definida no navegador

### Problema
A regra que o usuário enxerga é imposta apenas em React. Um `POST /api/backup` feito fora da interface — com curl, num script de automação, ou por qualquer código que passe a chamar a API depois — produz um backup válido protegido por 8 caracteres. E esse backup é o bundle completo: banco de pacientes, prontuários de sessão, anexos e, hoje, o pepper mestre (F3). A própria base de código já discorda de si mesma: `MIN_AUTO_BACKUP_PASSWORD_LEN = 12` em config.rs para a senha dos backups automáticos, contra `MIN_BACKUP_PASSWORD_LEN = 8` para o manual. E em nenhum dos dois caminhos há qualquer exigência além do comprimento.

### Evidência
- `src/pages/Settings.tsx:55`
- `src/pages/Settings.tsx:104`
- `src/pages/Settings.tsx:154`
- `src-tauri/src/features/backup.rs:25`
- `src-tauri/src/features/backup.rs:88-95`
- `src-tauri/src/config.rs:12`

```
// src/pages/Settings.tsx:55 — o que o usuário vê
if (pw.length < 12) {
  setPasswordError("Minimo 12 caracteres para backup com senha.");
  return;
}

// src-tauri/src/features/backup.rs:25 e 88-95 — o que o servidor aceita
const MIN_BACKUP_PASSWORD_LEN: usize = 8;
````

````
...
if let Some(pass) = password {
    if pass.chars().count() < MIN_BACKUP_PASSWORD_LEN {
        return Err(AppError::bad_request(format!(
            "A senha do backup deve ter no mínimo {} caracteres.", MIN_BACKUP_PASSWORD_LEN)));

// src-tauri/src/config.rs:12 — a senha automática, por outro lado, exige 12
pub const MIN_AUTO_BACKUP_PASSWORD_LEN: usize = 12;
```

### Impacto
Backups contendo o prontuário completo podem ser gerados com senha muito mais fraca do que a aplicação promete, reduzindo o custo de um ataque offline sobre o arquivo — e, por tabela, sobre a chave-raiz que ele carrega.

**Condições de exploração:** Explorável por qualquer chamada à API fora da interface. Não requer configuração insegura.
### Correção sugerida

Elevar `MIN_BACKUP_PASSWORD_LEN` ao mesmo valor da constante de config.rs — ou, melhor, importar `MIN_AUTO_BACKUP_PASSWORD_LEN` nos dois caminhos, para que não voltem a divergir — e acrescentar uma checagem de qualidade além do comprimento.
### Critérios de aceite

- [ ] Existe uma única constante de mínimo compartilhada pelos dois caminhos de backup
- [ ] `POST /api/backup` com senha de 8 a 11 caracteres retorna 400
- [ ] Teste Rust cobrindo a rejeição no limite (11 recusada, 12 aceita)
- [ ] A mensagem de erro do servidor bate com a exibida na tela de Configurações

---
_Aberto a partir do relatório de auditoria de segurança de 31 de agosto de 2026 (achado F4, versão 1.0.19, commit 6ed2d6a)._

> **Situação:** Corrigido — os dois caminhos de backup compartilham a constante de 12 caracteres.
````

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

````
## [Segurança] Chave de API Firebase hardcoded no histórico do git

**Labels:** `security, media, secrets`
**Severidade:** Média
**Categoria:** 4. Chaves expostas

### Problema
É o padrão exato de "default público que vira segredo real quando a variável não é sobrescrita": o `||` garante que a chave literal entra no bundle sempre que `VITE_FIREBASE_API_KEY` não estiver definida. O arquivo não existe mais no HEAD — a autenticação migrou para local, em Rust — mas `git log -S` confirma os dois commits e `git merge-base --is-ancestor` confirma que ambos são alcançáveis a partir de `main`: qualquer clone do repositório público os traz junto. Uma chave de API Web do Firebase é, por projeto, um identificador destinado ao cliente e não uma credencial de acesso — é por isso que este achado é médio e não crítico. Ainda assim ela expõe o projeto `atendemente-432ab` a abuso da API Identity Toolkit (criação de contas, consumo de cota) e a leitura de Firestore/Storage `na medida em que as regras de segurança daquele projeto forem permissivas` — o que não é verificável a partir deste repositório.

### Evidência
- `src/lib/auth.ts — commit ad4260f:1-6`
- `src/lib/auth.ts — commit ac6b0c7:1-6`

```
// src/lib/auth.ts:1-6 — em ad4260f e ac6b0c7, ambos ancestrais de main
const apiKey = import.meta.env.VITE_FIREBASE_API_KEY || "AIzaSyCMGKi2aHlRpOynxpjhb_7VZMMMqRaUa4Y";
const authDomain = import.meta.env.VITE_FIREBASE_AUTH_DOMAIN || "atendemente-432ab.firebaseapp.com";
const projectId = import.meta.env.VITE_FIREBASE_PROJECT_ID || "atendemente-432ab";
````

````
const storageBucket = import.meta.env.VITE_FIREBASE_STORAGE_BUCKET ||
"atendemente-432ab.firebasestorage.app";
const messagingSenderId = import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID || "416468948741";
const appId = import.meta.env.VITE_FIREBASE_APP_ID || "1:416468948741:web:f08ce79a49801f1fc3496c";
```
### Impacto

Superfície residual num projeto Firebase que a aplicação atual não usa mais. Se o projeto seguir ativo com regras abertas, permite acesso não autenticado aos recursos dele.
**Condições de exploração:** A exploração depende inteiramente das regras de segurança do projeto Firebase e de ele continuar ativo. A presença do segredo no histórico é fato verificado.

### Correção sugerida
Verificar no console do Firebase se o projeto `atendemente-432ab` ainda existe. Se sim: revogar/rotacionar a chave, restringi-la por aplicativo e revisar as regras de Firestore e Storage; se o projeto não é mais usado, desativá-lo. Reescrever o histórico com `git filter-repo` é possível, mas num repositório público já clonado o valor deve ser tratado como comprometido de qualquer forma — desativar o projeto é a correção que de fato fecha o risco.

### Critérios de aceite
- [ ] Confirmado o estado do projeto `atendemente-432ab` (ativo ou desativado)
- [ ] Se ativo: chave rotacionada e restrita, regras de Firestore/Storage revisadas e documentadas
- [ ] Nenhum novo default literal de credencial no padrão `env || "valor"` no código-fonte (verificação em CI)
- [ ] Decisão sobre reescrita de histórico registrada, com justificativa

---
_Aberto a partir do relatório de auditoria de segurança de 31 de agosto de 2026 (achado F5, versão 1.0.19, commit 6ed2d6a)._
> **Situação:** Resolvido pelo usuario — o projeto Firebase `atendemente-432ab` foi excluido, que era a correcao que de fato fechava o risco.
````

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

````
## [Segurança] CSP da webview permite script-src 'unsafe-inline'

**Labels:** `security, media, csp`
**Severidade:** Média
**Categoria:** 5. Inputs sem tratamento (XSS)

### Problema
O comentário em middleware.rs:25-28 registra corretamente que "a webview honra a CSP da configuração" — e é exatamente por isso que a política válida no produto instalado é a mais fraca das duas. Com `'unsafe-inline'` em `script-src`, qualquer injeção de HTML que venha a existir executa; sem ele, a mesma injeção seria inerte. A CSP da webview também não fecha `object-src`, `base-uri` nem `form-action`, que a do servidor fecha, e o teste `csp_locks_down_the_dangerous_directives` (middleware.rs:84-104) valida apenas a política do servidor — a divergência não é detectada por nenhum teste. Registre-se: `não foi encontrado nenhum sink de XSS no frontend` (zero ocorrências de innerHTML, dangerouslySetInnerHTML, eval ou new Function), de modo que isto é o enfraquecimento de uma defesa em profundidade, não uma injeção explorável hoje. O que a torna relevante é o conteúdo renderizado: nome, anotações, histórico clínico e medicações são texto livre digitado pelo usuário ou importado de CSV.

### Evidência
- `src-tauri/tauri.conf.json:25`
- `src-tauri/src/middleware.rs:29-43`
- `src-tauri/src/middleware.rs:84-104`
```
// src-tauri/tauri.conf.json:25 — a política que a webview do Tauri aplica
"csp": "default-src 'self'; connect-src 'self' http://localhost:* http://127.0.0.1:* ipc: asset: ws:;
        style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
        font-src 'self' https://fonts.gstatic.com; img-src 'self' data: blob:;
        script-src 'self' 'unsafe-inline' asset:"
                        ^^^^^^^^^^^^^^^^
````

````
// src-tauri/src/middleware.rs:37 — a política que o servidor envia
             script-src 'self'; \
             object-src 'none'; \
             base-uri 'none'; \
             form-action 'none'; \
```

### Impacto
A camada que conteria um XSS futuro está desativada precisamente no artefato distribuído aos usuários. Também não há nenhuma biblioteca de sanitização no projeto, caso um sink venha a ser introduzido.

**Condições de exploração:** Enfraquecimento de defesa em profundidade: só produz impacto direto se um sink de HTML for introduzido no frontend.
### Correção sugerida

Remover `'unsafe-inline'` de `script-src` no tauri.conf.json e alinhar as duas políticas, acrescentando `object-src 'none'`, `base-uri 'none'` e `form-action 'none'`. Estender o teste de middleware.rs para ler o tauri.conf.json e exigir das duas políticas o mesmo conjunto mínimo de diretivas, para que não voltem a divergir.
### Critérios de aceite

- [ ] `script-src` em `tauri.conf.json` não contém `'unsafe-inline'`
- [ ] `object-src 'none'`, `base-uri 'none'` e `form-action 'none'` presentes nas duas políticas
- [ ] Teste automatizado compara as diretivas de `tauri.conf.json` com as de `middleware.rs`
- [ ] O app abre e opera normalmente com a política endurecida (verificado no build de release)
---
_Aberto a partir do relatório de auditoria de segurança de 31 de agosto de 2026 (achado F6, versão 1.0.19, commit 6ed2d6a)._

> **Situação:** Corrigido — `unsafe-inline` removido de `script-src` no tauri.conf.json, com teste que compara as duas politicas.
````

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

````
## [Segurança] /auth/recover permite enumeração de contas antes do rate limit

**Labels:** `security, baixa`
**Severidade:** Baixa
**Categoria:** 2. Regra definida no navegador

### Problema
O limite de 3 tentativas por 15 minutos é aplicado sobre o `user_id`, que só existe depois de a conta ter sido encontrada. Consultas com e-mail inexistente, portanto, nunca incrementam contador nenhum: dá para varrer endereços indefinidamente e distinguir o 404 ("Nenhuma conta encontrada com este email.") do 401 ("Chave de recuperação inválida."). O contraste com o resto do arquivo é o que torna isto um achado e não uma escolha: o `login` foi cuidadosamente protegido contra o mesmo problema, gastando um Argon2 sobre um hash dummy quando a conta não existe (auth_service.rs:16-23 e 243-247) e devolvendo sempre "Email ou senha inválidos.". O caminho de recuperação não recebeu o mesmo tratamento. O servidor escuta apenas em loopback e o CORS por predicado impede que um site qualquer leia a resposta, o que mantém a severidade baixa.

### Evidência
- `src-tauri/src/auth/mod.rs:255-268`
- `src-tauri/src/auth/auth_service.rs:357-365`

```
// auth/mod.rs:255-268 — o lookup por e-mail roda ANTES do enforce_rate_limit
let user_id = match (&input.user_id, &input.email) {
    (Some(uid), _) if !uid.is_empty() => uid.clone(),
    (_, Some(email)) if !email.is_empty() => {
        auth_service::find_user_id_by_email(&state.auth_db, email)
            .await
            .map_err(|e| AppError::not_found(e))?      // <- o 404 distingue a conta
    }
````

```
    _ => return Err(AppError::bad_request("Informe user_id ou email.")),
};

crate::rate_limit::enforce_rate_limit(          // <- só agora
    &state.auth_db, "auth:password-reset", &user_id, 3, 900_000,
).await?;

// auth_service.rs:364
.ok_or_else(|| "Nenhuma conta encontrada com este email.".to_string())
```

Impacto
Confirmação de quais endereços de e-mail têm conta na máquina, sem custo e sem limite. Útil como etapa preparatória de um ataque local ao código de recuperação.

Condições de exploração: Requer alcançar a API em 127.0.0.1:3001, isto é, código já em execução na máquina do usuário. O servidor não escuta na rede.
Correção sugerida

Aplicar o rate limit sobre o identificador `recebido` — o e-mail informado — antes de resolvê-lo, e devolver a mesma resposta para conta existente e inexistente, no mesmo espírito do que já é feito no login.
Critérios de aceite

- [ ] `enforce_rate_limit` é chamado antes de `find_user_id_by_email`, com o e-mail como identificador
- [ ] A resposta para e-mail inexistente é indistinguível da resposta para código de recuperação errado (mesmo status e mesma mensagem)
- [ ] Teste Rust cobrindo os dois casos e comparando as respostas

---
Aberto a partir do relatório de auditoria de segurança de 31 de agosto de 2026 (achado F7, versão 1.0.19, commit 6ed2d6a).

> **Situação:** Corrigido — o rate limit passou a ser aplicado sobre o identificador recebido, antes do lookup, e a resposta e a mesma para conta inexistente e codigo errado.

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

[Segurança] Content-Type de download vem do MIME declarado, sem revalidação

Labels: `security, baixa`
Severidade: Baixa
Categoria: 5. Inputs sem tratamento (XSS)

Problema
`create_upload_session` grava `input.mime_type` sem validar; quem valida o MIME contra a extensão e contra os magic bytes reais é `validate_file_content`, chamada apenas em `confirm_upload` (files.rs:292). Mas `download_file` não exige confirmação — a única condição é `deleted_at IS NULL`. Uma sessão de upload criada e escrita, porém nunca confirmada, fica baixável com um `Content-Type` arbitrário escolhido pelo chamador. Três coisas seguram o impacto, e por isso a severidade é baixa: o `Content-Disposition: attachment` na linha seguinte impede a renderização inline; a extensão já foi restrita a pdf/docx/png/jpg/jpeg em `validate_file_metadata` (files.rs:46-57); e o app é monousuário, de modo que atacante e vítima são a mesma conta. O defeito real é de consistência: o modelo do código presume que todo registro legível passou pela validação, e há um caminho em que isso não vale.

Evidência
- `src-tauri/src/api/routes.rs:602`
- `src-tauri/src/features/files.rs:166-183`
- `src-tauri/src/features/files.rs:292`
- `src-tauri/src/features/files.rs:320-328`
```
// features/files.rs:179 — o MIME entra no banco como o cliente declarou
    .bind(&input.mime_type)
// features/files.rs:292 — a validação de magic bytes acontece só no confirm_upload
    validate_file_content(&data, &file.original_name, &file.mime_type)?;
```

```
// features/files.rs:320-323 — o download não revalida; exige apenas deleted_at IS NULL
    r#"SELECT * FROM record_files WHERE id = ? AND user_id = ? AND deleted_at IS NULL"#

// api/routes.rs:602 — e o valor vai direto para o cabeçalho
    (HeaderName::from_static("content-type"), file.mime_type),
```

Impacto
Um registro de arquivo não confirmado é servido com cabeçalho controlado pelo chamador. Sem impacto direto hoje, graças ao `attachment` e à whitelist de extensões.

Condições de exploração: Requer uma sessão autenticada da própria conta e um upload deixado deliberadamente sem confirmar.
Correção sugerida

Marcar a confirmação no banco — uma coluna `confirmed_at` — e exigi-la em `download_file` e `list_files_by_appointment`. Alternativa mais simples: ignorar o MIME armazenado na resposta e derivar o `Content-Type` dos magic bytes do conteúdo decifrado, no momento do download.
Critérios de aceite

- [ ] Arquivo não confirmado não é baixável (ou o `Content-Type` é derivado do conteúdo real)
- [ ] `list_files_by_appointment` não lista uploads não confirmados
- [ ] Teste Rust cobrindo o download de um registro criado mas não confirmado

---
Aberto a partir do relatório de auditoria de segurança de 31 de agosto de 2026 (achado F8, versão 1.0.19, commit 6ed2d6a).

> **Situação:** Corrigido — o `Content-Type` do download e derivado dos magic bytes do conteudo decifrado.

--- FIM ISSUE 8 ---

--- ISSUE 9 ---

[Segurança] Sem validação de tamanho de campo no servidor (crate validator não usada)

Labels: `security, baixa`
Severidade: Baixa
Categoria: 2. Regra definida no navegador

Problema
`AppError` tem uma variante `Validation` pronta para `ValidationErrors` (errors.rs:148-165) e a crate está no Cargo.toml, mas nenhum struct de entrada carrega atributos `#[validate]` — a validação de forma existe só em Zod, no cliente. Na prática, um `POST /api/patients` direto aceita um `admin_notes` de vários megabytes, limitado apenas pelo `DefaultBodyLimit` global de 20 MB (lib.rs:371). As regras que `de fato` importam para segurança estão no servidor e foram verificadas: nome com no mínimo 3 caracteres (patients.rs:620), senha com no mínimo 8 (auth_service.rs:121), fim depois do início (appointments.rs:225), recorrência limitada a 52 ocorrências (appointments_recurrence.rs:18-21). O que falta é o teto de tamanho.

Evidência
- `src-tauri/src/db/models.rs:1-375`
- `src-tauri/Cargo.toml:56`
- `src/lib/schemas.ts:14-24`
```
# src-tauri/Cargo.toml:56 — a dependência existe
validator = { version = "0.19", features = ["derive"] }
// src-tauri/src/db/models.rs — nenhuma ocorrência de #[validate] em todo o arquivo
pub struct CreatePatientInput {
    pub full_name: String,
    pub health_history: Option<String>,     // sem limite
    pub medications_in_use: Option<String>, // sem limite
    pub admin_notes: Option<String>,        // sem limite

// src/lib/schemas.ts:14-24 — a única validação de forma é a do cliente
export const patientSchema = z.object({
  full_name: z.string().min(2, "Nome deve ter no mínimo 2 caracteres"),
  health_history: z.string().optional(),
```

````
### Impacto

Crescimento descontrolado do banco e da memória por requisição, a partir de chamadas fora da interface. Sem impacto sobre confidencialidade ou integridade.
**Condições de exploração:** Requer chamada à API fora da interface. O limite global de corpo de 20 MB continua valendo.

### Correção sugerida
Aplicar `#[validate(length(max = ...))]` nos structs de entrada de `models.rs` e chamar `input.validate()?` no início de cada handler que os recebe — a variante de erro já está implementada e formatada.

### Critérios de aceite
- [ ] Structs de entrada em `models.rs` têm limites de comprimento declarados
- [ ] Handlers chamam `validate()` antes de persistir
- [ ] `POST /api/patients` com campo acima do limite retorna 400 com o corpo de erro de validação
---
_Aberto a partir do relatório de auditoria de segurança de 31 de agosto de 2026 (achado F9, versão 1.0.19, commit 6ed2d6a)._
> **Situação:** Corrigido — os sete structs de entrada ganharam `#[validate]` e os handlers chamam `validate()`.
````

--- FIM ISSUE 9 ---

--- ISSUE 10 ---

````
## [Segurança] App carrega Google Fonts da internet e a CSP foi afrouxada para isso

**Labels:** `security, baixa, privacidade`
**Severidade:** Baixa
**Categoria:** 5. Inputs sem tratamento (XSS)

### Problema
Um aplicativo desktop de prontuário psicológico, que o README descreve como local, faz uma requisição a servidores do Google a cada abertura. O conteúdo do prontuário não vaza, mas o padrão de uso sim: endereço IP do consultório e horário de cada sessão de trabalho ficam registrados num terceiro que não é operador contratado para esse tratamento. É também a razão de a CSP da webview precisar liberar dois domínios externos (F6), e uma dependência de rede num app que deveria funcionar sem ela: sem internet, a tipografia cai para a fonte de sistema.

### Evidência
- `src/style.css:1`
- `src-tauri/tauri.conf.json:25`

```
/* src/style.css:1 */
@import url("https://fonts.googleapis.com/css2?family=Manrope:wght@400;500;600;700;800&
             family=Newsreader:ital,wght@0,400;0,500;0,600;0,700;1,400&display=swap");
// src-tauri/tauri.conf.json:25 — a CSP precisou ser aberta por causa disso
"style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
 font-src 'self' https://fonts.gstatic.com;"
```
### Impacto

Vazamento de metadados de uso para terceiro e dependência de rede desnecessária. Relevante para o registro de operadores exigido pela LGPD.
**Condições de exploração:** Ocorre em toda execução com rede disponível. Não requer configuração especial.

### Correção sugerida
Baixar os arquivos `.woff2` das duas famílias, servi-los do próprio bundle com `@font-face` local, e então remover `fonts.googleapis.com` e `fonts.gstatic.com` da CSP.

### Critérios de aceite
- [ ] As fontes são carregadas de arquivos locais do bundle
- [ ] `src/style.css` não contém `@import` de host externo
- [ ] A CSP de `tauri.conf.json` não lista `fonts.googleapis.com` nem `fonts.gstatic.com`
````

```
- [ ] O app abre sem rede e renderiza com a tipografia correta

---
_Aberto a partir do relatório de auditoria de segurança de 31 de agosto de 2026 (achado F10, versão 1.0.19, commit 6ed2d6a)._
> **Situação:** Corrigido — as fontes sao servidas do bundle e os dois dominios sairam da CSP.
```

--- FIM ISSUE 10 ---

--- ISSUE 11 ---

````
## [Segurança] Rejeitar peppers degenerados na inicialização

**Labels:** `security, informativa`
**Severidade:** Informativa
**Categoria:** 4. Chaves expostas

### Problema
Registrado por completude da varredura, não como falha. Nenhum desses valores é segredo de produção: o pepper zerado do CI roda contra bancos em `mktemp -d` e existe justamente para que o smoke test não dependa do cofre de credenciais do runner; as senhas de e2e criam contas descartáveis. O único ponto que merece registro é que `load_or_generate_pepper` (config.rs:146-180) aceita um pepper de 32 bytes nulos sem qualquer objeção — não existe validação de inicialização que recuse valores notoriamente fracos ou de exemplo. Vale também anotar o acerto correspondente: a variável de ambiente `nunca` é persistida no cofre (o comentário em config.rs:147-154 documenta a regressão que motivou a mudança), então um pepper de teste não pode mais substituir silenciosamente o real.

### Evidência
- `.github/workflows/ci.yml:102`
- `.github/workflows/ci.yml:136`
- `e2e/fixtures/auth.ts:21`
- `e2e/fixtures/test-data.ts:89`
- `scripts/seed-demo.ts — commit a05f804:25`

```
# .github/workflows/ci.yml:102
export MASTER_PEPPER=0000000000000000000000000000000000000000000000000000000000000000

# .github/workflows/ci.yml:136
-d '{"email":"smoke@test.local","password":"senha123456","full_name":"Smoke"}'
// e2e/fixtures/auth.ts:21
const password = "test123456";
// scripts/seed-demo.ts:25 (arquivo já removido; permanece no commit a05f804)
const demoPassword = "AtendeMente123!";
```
### Impacto

Nenhum sobre a instalação do usuário final.
**Condições de exploração:** Não aplicável.

### Correção sugerida
Opcionalmente, recusar peppers degenerados — todos os bytes iguais — em `load_or_generate_pepper` quando o processo não estiver num modo de teste declarado, para que um valor de exemplo copiado da documentação não vire chave real.

### Critérios de aceite
- [ ] `load_or_generate_pepper` recusa um pepper com todos os bytes iguais fora de modo de teste
- [ ] Teste Rust cobrindo a recusa

---
_Aberto a partir do relatório de auditoria de segurança de 31 de agosto de 2026 (achado F11, versão 1.0.19, commit 6ed2d6a)._

> **Situação:** Corrigido — peppers degenerados sao recusados no boot e o `hex_decode` do config nao entra mais em panico com entrada multi-byte.
````

--- FIM ISSUE 11 ---